

zrux

A personal context engine for a founder. Ingests context from many sources, stores it centrally, and answers grounded, cited questions about your work.

Deep reference for the architecture call. Read the italic summary at the top of each section out loud; drill into the detail underneath when asked. The last section is a cheat sheet of likely questions with ready answers.

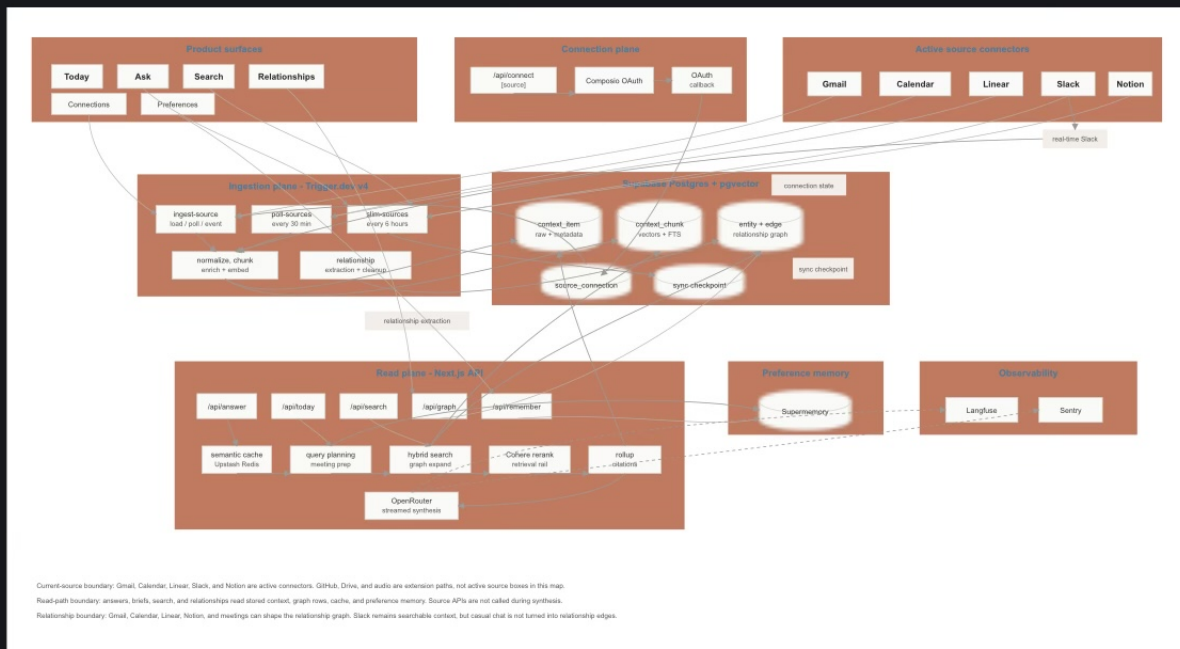
The whole system in one sentence: *An ingestion plane reads your sources on a schedule and writes normalized, embedded context into Postgres; an answer plane reads only Postgres and calls an LLM to synthesize a grounded, cited reply. The two planes share nothing but the database. That separation is the spine of everything below.*

1. The 60-second pitch (read this first)

- **What it is:** a context engine, not a live-API chatbot. It answers questions like "what should I focus on today?", "summarize investor activity this week", "which tasks are blocked?"
- **Sources today:** Gmail, Google Calendar, Linear, Slack, Notion (five live connectors), all behind one uniform Connector contract, OAuth managed by Composio.
- **Ingestion is background:** Trigger.dev jobs poll every 30 minutes, fetch, normalize, chunk, enrich, embed, and upsert into Postgres. It can be slow. Nobody is waiting on it.
- **Answering is fast and synchronous:** it never calls Gmail or Linear at answer time. It reads pre-ingested context out of Postgres and synthesizes.
- **Storage:** Supabase Postgres + pgvector. Hybrid search = dense vectors + keyword full-text, fused with Reciprocal Rank Fusion, then time-decayed.
- **Three memory layers:** L1 context engine (RAG), L2 relationship graph (people / companies / projects), L3 personalization (durable founder preferences via Supermemory).
- **Grounded and safe:** every claim is cited; the answer model is read-only with zero tools, which is the primary defense against prompt injection. If context is thin, it refuses.
- **Production-grade resilience:** semantic cache, circuit breaker with model fallback, graceful degradation, full Langfuse tracing, Sentry error reporting.

2. Your diagram, checked against the real code

Verdict: the diagram you attached is an accurate map of what is actually built. Every plane and nearly every box exists in the codebase. The only deltas are two cosmetic table-name differences and a handful of boxes that are deliberately not wired yet (audio, GitHub, Sentry-as-source). Cadences in the diagram match the cron schedules in the code exactly.



Box in your diagram	In the code?	Notes
Product surfaces: Today, Ask, Search, Relationships, Connections, Preferences	■ matches	All shipped as Next.js pages. "Preferences" lives inside the Ask screen.
Connection plane: /api/connect, Composio OAuth, OAuth callback	■ matches	Exactly the connect → Composio link → callback reconcile flow.
Active source connectors: Gmail, Calendar, Linear, Slack, Notion	■ matches	Five live connectors. "real-time Slack" arrow = the Slack webhook path.
Ingestion plane (Trigger.dev): ingest-source, poll every 30 min, slim every 6 hours, normalize/chunk/enrich+embed, relationship extraction	■ matches	Poll cron is <code>* / 30 * * * *</code> , slim cron is <code>15 * / 6 * * *</code> . Both exactly as drawn.

Supabase Postgres + pgvector: context_item, context_chunk, entity + edge, source_connection	■ matches	All four tables exist as drawn.
Diagram label "sync-checkpoint"	■■ naming	Real table is sync_state (per user_id + source incremental cursor). Same idea.
Diagram label "connection state"	■■ naming	Real table is source_connection (OAuth account + status). Same idea.
Read plane: /api/answer, /api/today, /api/search, /api/graph, /api/remember	■ matches	All five endpoints shipped.
Read plane internals: semantic cache (Upstash Redis), query planning, hybrid search + graph expand, Cohere rerank, rollup, OpenRouter streamed synthesis	■ matches	This is the answer pipeline, stage for stage (section 9).
Preference memory: Supermemory	■ matches	Layer 3. Backs /api/remember and reorders answers.
Observability: Langfuse, Sentry	■ matches	Langfuse traces every LLM + retrieval stage; Sentry catches errors, PII-safe.
(Not on diagram) GitHub, Sentry-as-source, Drive, voice memo / audio	■ not wired	Enum + stub only. Deliberate 48-hour cut. See section 14.

3. The two planes (the core mental model)

One idea to anchor the whole call: ingestion and answering are completely decoupled. They never call each other. They communicate only through Postgres.

Ingestion plane	Answer plane
When it runs Background, on a schedule (or webhook)	Synchronously, when you ask a question
Where it runs Trigger.dev jobs (long-running, retrieable)	Next.js API routes (streamed)
Where it calls source APIs? Gmail, Linear, etc. via Composio	Never. Reads only Postgres
Speed budget Can take minutes; nobody waits	Must feel instant; streamed token by token
What it writes text_item, context_chunk, entity, edge	Nothing source-side; only cache + preference learning

Why this matters (say this if asked "why not just call Gmail when I ask?"): calling live APIs at answer time would be slow, rate-limited, and fragile, and would couple your answer latency to five third-party services being up. Pre-ingesting decouples answer speed from source availability, lets us embed and index ahead of time, and makes answers reproducible.

4. Three memory layers

In one breath: Layer 1 is the searchable record of everything (RAG over your content). Layer 2 is the typed graph of who and what (people, companies, projects, and how they relate). Layer 3 is what you personally care about (durable preferences that shape ordering).

Layer	What	Where it lives	Used for
-------	------	----------------	----------

L1 Context Engine	Cross-source RAG: every email, issue, message, doc, meeting, chunked + embedded	Supabase: <code>context_item</code> , <code>context_chunk</code> (pgvector)	Point-in-time grounded answers with citations
L2 Relationship Graph	Typed entities + edges: <code>invested_in</code> , <code>works_with</code> , <code>introduced_by</code> , <code>decided</code>	Supabase: <code>entity</code> , <code>edge</code>	"Who is this person, what connects them" relational queries; Relationships screen
L3 Personalization	Cross-session founder profile and preferences	Supabase: Supermemory (per-tenant container)	Reordering / emphasis of answers. Never adds facts.

5. Data model deep-dive

Everything is in Supabase Postgres with the pgvector extension. Every table carries `user_id` and every query is scoped by it first (multi-tenancy).

`context_item` — one row per source item

An email, Linear issue, Slack message, Notion doc, calendar event, etc. Holds title, author, url, status, metadata (JSONB), the **raw payload** (episodic ground truth, re-processable), `is_deleted` (flipped by the slim job), and crucially **both** `source_created_at` and `source_updated_at`. Unique on (`user_id`, `source`, `external_id`) so re-ingestion is idempotent.

Why two timestamps (likely question): "what is happening" questions sort by *updated*; "what was decided in Q1" questions sort by *created*. A single `occurred_at` would collapse those two very different intents.

`context_chunk` — the retrieval target, hash-partitioned by `user_id`

Long documents are split into chunks. Each chunk stores its `content` (provenance line + optional gloss + body), a 1536-dim `embedding`, and a generated `fts` tsvector for keyword search. It is **hash-partitioned by `user_id` into 8 partitions (p0–p7)** from the very first migration. Primary key is (`user_id`, `id`) because the partition key must sit in the PK.

Indexes: HNSW on `embedding` (vector cosine), GIN on `fts` (full-text), and a compound (`user_id`, `source`, `source_updated_at` desc) for time filters. A second HNSW sits on `context_item.summary_embedding` for a future two-tier index.

`entity` + `edge` — the relationship graph (Layer 2)

`entity` is people / companies / projects, keyed canonically on `email` when present (unique per user), with a `pg_trgm` GIN index on `name` for fuzzy matching. `edge` is typed relations between two entities (`subject_id`, `object_id`, `relation`, `confidence`, `source_item`, `occurred_at`), unique on (`user_id`, `subject_id`, `relation`, `object_id`, `source_item`) so re-extraction does not create duplicates.

Supporting tables

- `sync_state` — per (`user_id`, `source`) incremental bookmark: `last_successful_sync_at` + opaque `cursor`. (This is the diagram's "sync-checkpoint".)
- `source_connection` — per (`user_id`, `source`) OAuth account + status (initiated / active / error). (The diagram's "connection state".)
- `briefing` — durable per-tenant cache of the precomputed Today briefing, with a TTL.

Multi-tenancy (likely question: "how do you stop tenant A reading tenant B?")

Two layers. **Primary:** every application query filters on `user_id` first. **Secondary:** Row-Level Security policies (`user_id = auth.uid()`) as defense-in-depth. Note: `auth` is NextAuth, so server-side reads use the service-role client (which bypasses RLS by design) and rely on the app-level scoping; RLS still catches any JWT/anon path.

6. Ingestion pipeline, end to end

In one breath: a scheduled job asks a connector for items, we persist the raw payload, normalize it, chunk long text, prepend a provenance line (and optionally one LLM-written sentence of context), embed it, upsert item + chunks, then for high-signal sources extract relationship triples and resolve them into the entity graph.

The Connector contract (one interface, every source)

```
interface Connector { source: SourceName load(ctx): AsyncIterable<RawItem> // full bulk,
first connect poll(ctx, since): AsyncIterable<RawItem> // incremental by cursor slim(ctx):
AsyncIterable<ExternalId> // ids only, deletion detection handleEvent?(payload):
AsyncIterable<RawItem> // optional webhook }
```

Composio supplies OAuth and the authenticated fetch *inside* load/poll/slim. The contract is ours, so swapping Composio for Nango is a one-file change at the same seam. Five connectors implement it today: **Gmail, Calendar, Linear, Slack, Notion.**

The steps (in order)

1. **Trigger:** scheduled poll, load on first connect, or a webhook event.
2. **Fetch + persist raw:** the untouched source payload is stored in `context_item.raw` as episodic ground truth so we can reprocess later without re-fetching.
3. **Normalize:** map to a `context_item` row, always setting both timestamps.
4. **Chunk:** structured items (Linear issues, calendar events) stay one chunk; long unstructured text (emails, Notion docs) splits on paragraph boundaries (~1500 chars, 150 overlap).
5. **Enrich:** a deterministic provenance line `[Source][date][author]` is always prepended; an optional one-sentence LLM "gloss" is added only for long/unstructured content and only when enabled (cost control). Structured items skip the gloss.
6. **Embed:** OpenAI `text-embedding-3-large` at 1536 dims, batched per item.
7. **Upsert:** item + chunks. Chunk replace is delete-then-insert in one retry block so a network blip cannot leave duplicates.
8. **Triple extraction (gated):** only for high-signal sources (gmail, calendar, notion, linear, meetings). Slack and Sentry are excluded. Bulk/promotional Gmail and automated senders are filtered out.
9. **Entity resolution:** resolve each extracted endpoint to an entity, then upsert entity + edge.

Every item is processed in isolation, so one bad item never aborts a 90-day backfill. Enrichment and extraction are best-effort: if they fail, the embedding still lands.

Entity resolution rules (say "we always prefer a missed merge over a wrong merge")

1. Email is the canonical key. Match on email first, always.
2. No email: conservative pg_trgm fuzzy match on normalized name within the same type (threshold ~0.45 — merges "Sarah" with "Sarah Chen", keeps "Sarah Chen" and "Sarah Connor" apart).
3. Companies: name + domain.
4. Unresolvable: create a new provisional entity rather than risk a wrong merge.

Trigger.dev jobs

- **ingest-source** — the worker, runs in mode load / poll / event. Max 10 min, 5 retries with backoff. Event mode does not advance the poll cursor, so webhooks never cause polls to skip data.
- **poll-sources** — cron `* / 30 * * * *` (every 30 min): enqueue an incremental poll for every active connection.
- **slim-sources** — cron `15 * / 6 * * * *` (every 6 h): list live ids, diff against stored ids, flip `is_deleted` on vanished items. **Safety rail:** an empty live set (likely a fetch failure) refuses to mass-delete.

7. OAuth / connection lifecycle

```
POST /api/connect/[source] -> Composio.connectedAccounts.link() -> returns redirectUrl
-> write source_connection row (status = "initiated") -> user completes OAuth at
provider GET /api/oauth/callback -> reconcile "initiated" rows against live Composio
status -> if ACTIVE: mark "active" + enqueue ingest (mode = load) -> if
failed/expired/timed-out: mark "error" DELETE /api/connect/[source] -> revoke Composio
account, remove row -> historical context_item rows stay (disconnect stops future
syncs, does not erase history)
```

Abandoned flows (user hits back, callback never fires) are healed by a reconcile sweep that re-checks Composio status. The Slack **webhook path** (the "real-time Slack" arrow on your diagram) HMAC-verifies the request, keeps only human messages, resolves the workspace to a user, dedupes on Slack's event id, and enqueues to Trigger.dev. Ingestion never runs inline in the route.

8. Answer pipeline, stage by stage (the headline)

In one breath: *embed the question, check the cache, understand the query with one LLM call, run hybrid search over the tenant's chunks, rerank, drop the weak ones, expand the graph, roll chunks up to items, assemble a numbered cited context, and stream a grounded synthesis that cites every claim or refuses.*

```
QUESTION -> [0] embed question once (reused for cache + search) -> [0a] semantic cache: cosine >= 0.95 vs cached queries -> HIT short-circuits everything -> [1] query understanding (1 LLM call -> plan JSON) -> [1a] meeting-prep enrichment (only if intent = meeting_prep) -> [2] hybrid_search() (dense + keyword, RRF, recency decay) -> [2b] Cohere rerank 3.5 (graceful degrade if unavailable) -> [2c] retrieval rail (drop chunks below 0.3x the top score) -> [3] graph expansion (resolve named entities, pull one-hop edges) [parallel] -> [L3] personalization profile block [parallel] -> [5] rollup chunks -> parent items (dedupe, diversify, cap 8) -> [7] assemble: L3 profile + L2 facts + numbered cited items -> [8] stream synthesis (read-only model, cite every claim, refuse if thin) -> write answer to cache; enqueue preference learning (fire-and-forget)
```

Stage 1 — query understanding

One structured LLM call turns the raw question into a plan: `semantic_query`, `keyword_terms`, `sources[]`, `after/before`, `type`, `status`, `entities[]`, `intent`, `time_basis` (updated vs created), and `recency_weight` (0 for a precise lookup, ~0.3 for a daily briefing). Intent is one of eight values (`daily_briefing`, `meeting_prep`, `followup_detection`, `blocker_scan`, `investor_summary`, `company_summary`, `cross_source`, `lookup`).

Stage 1a — meeting prep

If intent is `meeting_prep`, we find the next calendar meeting, resolve its participants to entities, rewrite the plan to pull their history (clearing time filters), and force the meeting itself to be cited.

Stage 2 — hybrid search (the heart of retrieval)

A single Postgres function runs two retrievals over the tenant's filtered chunks: a dense vector search and a keyword full-text search. It fuses them with Reciprocal Rank Fusion, then applies an exponential recency multiplier. For broad intents it stratifies per source so a noisy inbox cannot crowd out everything else; if filters return too little, it relaxes them and retries.

```
create function hybrid_search(p_user_id, p_query_embedding, p_query_text, p_sources,
p_after, p_time_basis, p_recency_weight, p_limit) ... filtered as ( select * from
context_chunk where user_id = p_user_id and (p_sources is null or source = any(p_sources))
and (p_after is null or ts >= p_after) ), vec as ( ... order by embedding <=>
```

```
p_query_embedding limit 50 ), kw as ( ... where fts @@ websearch_to_tsquery('english',  
p_query_text) limit 50 ), fused as ( select 1.0/(60+vec.rnk) + 1.0/(60+kw.rnk) as rrf from  
vec full outer join kw on vec.id = kw.id ) select ..., rrf * (1 + p_recency_weight *  
exp(-extract(epoch from (now()-ts)) / (30*86400.0))) as score order by score desc limit  
p_limit;
```

Important nuance to volunteer ("why exact KNN, not HNSW?"): because `filtered` is referenced twice, Postgres materializes it, so the vector CTE does an *exact* nearest-neighbor sort over that one tenant's chunks rather than an approximate HNSW index scan. At per-tenant scale that is both more accurate and plenty fast. The HNSW index only starts to matter at millions of vectors.

Stages 2b / 2c — rerank and rail

Cohere Rerank 3.5 re-scores the ~50 candidates with a cross-encoder. It is a quality upgrade, not a correctness requirement: if Cohere is unavailable, every hit gets score 0 and the pipeline continues. The **retrieval rail** then drops chunks scoring below 0.3x the top chunk's score. The threshold is *relative* (not a fixed floor) so it adapts to Cohere's per-query scale and never silently discards clearly-relevant items.

Stages 3, 5, 7 — graph, rollup, assemble

Graph expansion resolves named entities and pulls one-hop edges into human-readable facts (best effort, never fails the answer). Rollup deduplicates chunks to their parent item, keeps the best chunk each, diversifies by source for broad intents, and caps at 8 items. Assemble builds the final context: the Layer 3 founder-profile block first, then Layer 2 relationship facts, then the numbered, cited items.

Stage 8 — synthesis

A read-only model streams the answer. It must cite every factual sentence with `[n]`, may use the founder profile only to order and emphasize (never to add facts), and refuses plainly when the context is thin. On success the answer is written to the semantic cache and a preference-learning job is enqueued.

9. LLM gateway and resilience

Mechanism	How it works
Gateway	OpenRouter. Primary <code>anthropic/claude-sonnet-4-6</code> , fallback <code>anthropic/claude-haiku-4-5</code> .
Circuit breaker	Redis-backed, global. Trips OPEN after ~5 failures in 60s (only 5xx / network / timeout count; 4xx and 429 pass through). 30s cooldown, then HALF_OPEN probe.
Fallback + retry	Retry with backoff on the primary, then fall back to the secondary model.
Graceful degradation	If the breaker is open at answer time, return the already-retrieved cited context with a "summary temporarily unavailable" banner, still HTTP 200. The user gets sources, not an error page.
Semantic cache	Upstash Redis, per-tenant, cosine ≥ 0.95 , ~90-min TTL, capped at 200 entries. A hit skips the entire pipeline.
Fail-open everywhere	Cache, rerank, graph, and personalization all degrade quietly; none of them can fail an answer.
Briefing stagger	Proactive Today briefings are spread across a window through a bounded Trigger.dev queue, never a synchronous fan-out.

10. Security and injection defense

- **Primary:** the answer-time model is **read-only with zero tools**. A successful injection can only change answer text; it cannot act on anything.
- **Secondary:** strict data/instruction separation in the synthesis prompt. Retrieved content is data, never instructions.
- **Tertiary:** the retrieval rail drops distant chunks and caps item count, shrinking the injection surface.
- **Citations or refusal:** thin context returns an explicit refusal rather than a guess.

- Pattern scanning is treated as defense-in-depth only, never relied on (trivially bypassed by paraphrase).

11. Product surfaces

Screen	What it does	API
Today	Precomputed morning briefing, ranked by time-sensitivity (and your preferences). Served cache-first from the <code>briefing</code> table with inline fallback.	GET /api/today
Ask	Streamed grounded chat with inline [n] citation chips and expandable source cards. Add/forget preferences here.	POST /api/answer
Search	Corpus-wide hybrid keyword + semantic search with source filters. No synthesis, no time decay.	GET /api/search
Relationships	Strength-ranked contacts from a deterministic 8-factor interaction model (recency, frequency, reciprocity, responsiveness). Surfaces "losing touch" and "awaiting reply".	GET /api/graph
Connections	Live source management: connect / disconnect / reconnect, item counts, last sync, polled every few seconds.	/api/connect, /api/connections
Onboarding	Guided stepper that connects sources and unlocks the app once indexing produces items.	/api/connections/reconcile

Note: the Relationships strength score is computed deterministically from interaction metadata (no LLM). The Layer 2 entity/edge graph is the LLM-extracted side and powers graph expansion inside answers.

12. Observability and evaluation

- **Langfuse** runs on its own isolated OpenTelemetry provider (so it does not clobber Sentry's global one) and traces every LLM call and every retrieval stage, with secret masking before export. This is the replay log of the AI's decisions.
- **Sentry** catches exceptions and 5xx on server, edge, and client, configured PII-safe (`sendDefaultPii: false`, no local variables, replay masks all text).
- **Eval harness** seeds a fixture through the real ingestion pipeline (real embeddings) and uses an LLM judge for groundedness.

13. What is shipped vs. what is deliberately deferred

Shipped and working: 5 live connectors (Gmail, Calendar, Linear, Slack, Notion); full ingestion pipeline (normalize, chunk, enrich, embed, upsert, triple extraction, entity resolution); the complete 8-stage answer pipeline; all three memory layers; semantic cache; circuit breaker + model fallback; Cohere rerank; retrieval rail; graceful degradation; all six product surfaces; Langfuse + Sentry; eval harness. The three demo questions ("focus today", "investor activity this week", "blocked tasks") answer end to end with real citations.

Designed-in but not wired (honest cut lines for a 48-hour build):

- **GitHub, Sentry-as-a-source, Google Drive:** the source names and Composio auth slots exist, but there is no connector implementation yet. Adding one is a single file against the existing Connector contract.
- **Voice memos / audio (Deepgram Nova-3):** designed for diarized transcripts whose speaker turns become chunks and resolve against calendar participants, but the transcription step is not wired.
- **Two-tier doc index:** the `summary` and `summary_embedding` columns exist on `context_item` but are not yet populated.
- **Telegram bot:** a planned stretch delivery surface, not built.

Frame these as scope decisions, not gaps: the seams are already in place, so each is an additive change.

14. Key tradeoffs (have one line ready for each)

- **Composio over Nango:** chose Composio for velocity in 48 hours; the Connector contract abstracts it, so Nango is a one-file swap at the same seam.
- **Exact KNN over HNSW in hybrid_search:** deliberate and correct at per-tenant scale; more accurate; HNSW only matters at 10M+ vectors.
- **Modular monolith over microservices:** one codebase, one deploy, shared types; the only split that matters (read vs ingestion) is already done via the database boundary.

- **Supermemory over a hand-rolled founder_profile table:** removes build surface in a 48-hour window; a profile table is the production-hardened alternative.
- **Raw JSONB now, object-store later:** raw payloads live as JSONB in Postgres; the production lever is offloading to S3-class storage, same code path, one flag.

15. Likely questions and crisp answers

Q. What happens if the LLM provider goes down mid-question?

The circuit breaker trips after repeated 5xx/timeout failures. New questions then return the retrieved, cited context with a "summary temporarily unavailable" banner, still a 200. We also fall back from Sonnet to Haiku before that. The user always gets sources, never a blank error.

Q. Why don't you call Gmail or Linear when I ask a question?

Because that couples answer latency to five third-party APIs being up and rate-limit-friendly. We pre-ingest, embed, and index on a schedule, so answering is fast, reproducible, and resilient to a source being temporarily down.

Q. How is hybrid search actually ranked?

Two retrievals (dense vectors and keyword full-text) over this tenant's chunks, fused with Reciprocal Rank Fusion ($1/(60+\text{rank})$ summed), then multiplied by an exponential recency factor whose weight the query planner sets per intent.

Q. Why exact KNN instead of the HNSW index?

The filtered set is materialized and referenced twice, so Postgres does an exact sort over one tenant's chunks. At per-tenant scale that is more accurate and fast enough; HNSW only pays off at millions of vectors, which is why the index is there for later.

Q. How do you stop one long Notion doc from dominating the answer?

Chunk-to-item rollup keeps only the best chunk per parent item, broad intents interleave by source, and the context is capped at 8 items, so diverse sources survive.

Q. How is multi-tenancy enforced?

Every query filters on `user_id` first (primary), and Row-Level Security policies sit underneath as defense-in-depth. The chunk table is even hash-partitioned by `user_id`.

Q. What's your defense against prompt injection in retrieved content?

The answer model is read-only with zero tools, so an injection can at worst change wording, not take action. On top of that we separate data from instructions in the prompt and the rail trims distant chunks.

Q. How does "what should I know before my next meeting" work?

The planner detects meeting_prep intent, we look up the next calendar event, resolve its participants to entities, rewrite the query to pull their history and open items, and force the meeting itself into the citations.

Q. How does the system handle deletions in the source?

Every 6 hours a slim job lists live ids and flips is_deleted on anything that vanished, with a safety rail that refuses to mass-delete when the live set comes back empty (a likely fetch failure).

Q. How does personalization avoid making things up?

Layer 3 preferences can only reorder and emphasize. They are injected as a clearly-labeled profile block and the synthesis prompt forbids using them to add facts, so they can never turn a thin answer into a confident wrong one.